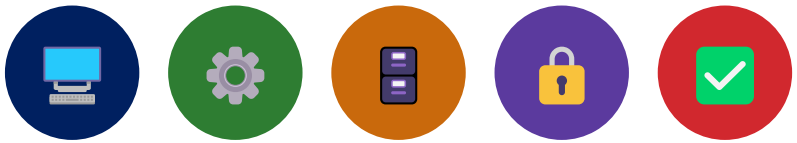
"Controllers talk HTTP. Services own the rules. Repositories own the data."

MODULE 25

Service and Repository Layers

Design, build, and test the Service and Repository layers that keep enterprise Spring applications clean, testable, and ready to scale.




TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

WHY LAYERED ARCHITECTURE MATTERS

Layered architecture organizes an application into clear, logical layers with well-defined responsibilities and boundaries.

It helps teams build robust, maintainable, and scalable applications that can evolve with business needs.

THE LAYERED ARCHITECTURE MODEL

Layer	Role
Presentation	Handles user requests and responses (Controllers, APIs).
Service	Contains business logic and orchestrates operations.
Repository	Handles data access and interacts with the database.
Database	Persists and manages application data.

KEY BENEFITS

Separation of Concerns — each layer has a single responsibility, reducing complexity.

Maintainability — changes in one layer have minimal impact on others.

Scalability — layers can be scaled independently based on demand.

Testability — layers can be unit tested in isolation for better quality.

Team Productivity — teams can work on different layers in parallel with clear contracts.

WITHOUT LAYERED ARCHITECTURE

Tight Coupling | **Difficult Maintenance** | **Hard to Test** | **Poor Scalability** | **Slow Delivery**

KEY TAKEAWAY Layered architecture brings structure, clarity, and consistency -- build it right, build it once, scale with confidence.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to understand, design, and implement Service and Repository layers following enterprise best practices.

Explain Layered Architecture — its importance and its role in building enterprise applications.

Describe the Pattern — the Controller-Service-Repository pattern and the flow of requests through each layer.

Define the Service Layer — explain how it manages business logic and application rules.

Implement Service Components — with a focus on reusability, testability, and maintainability.

Manage Service Dependencies — explain dependencies in the service layer and how to manage them effectively.

Design Repository Components — that provide clean, efficient, and secure data access.

Apply Repository Best Practices — including exception handling and performance.

Understand Layer Communication — how layers communicate and depend on each other.

Apply Enterprise Boundaries — identify and apply design boundaries for transactions, validation, and error handling.

Build a Layered Application — using Service and Repository layers following best practices.

KEY TAKEAWAY Mastering Service and Repository layers is essential for building scalable, maintainable, and enterprise-ready applications.

CONTROLLER-SERVICE-REPOSITORY PATTERN (1 OF 2)

The pattern organizes an application into three distinct layers with well-defined responsibilities and clear interactions.

"Each layer has a single focus and communicates only with the layer next to it. This separation leads to clean code, high maintainability, and easier testing."

Layer	Responsibility
Controller Layer	Handles HTTP requests and responses. Validates input and delegates to the Service.
Service Layer	Contains business logic and application rules. Orchestrates operations and calls the Repository.
Repository Layer	Handles data access and persistence. Abstracts and encapsulates database operations.
Database	Stores and manages application data.

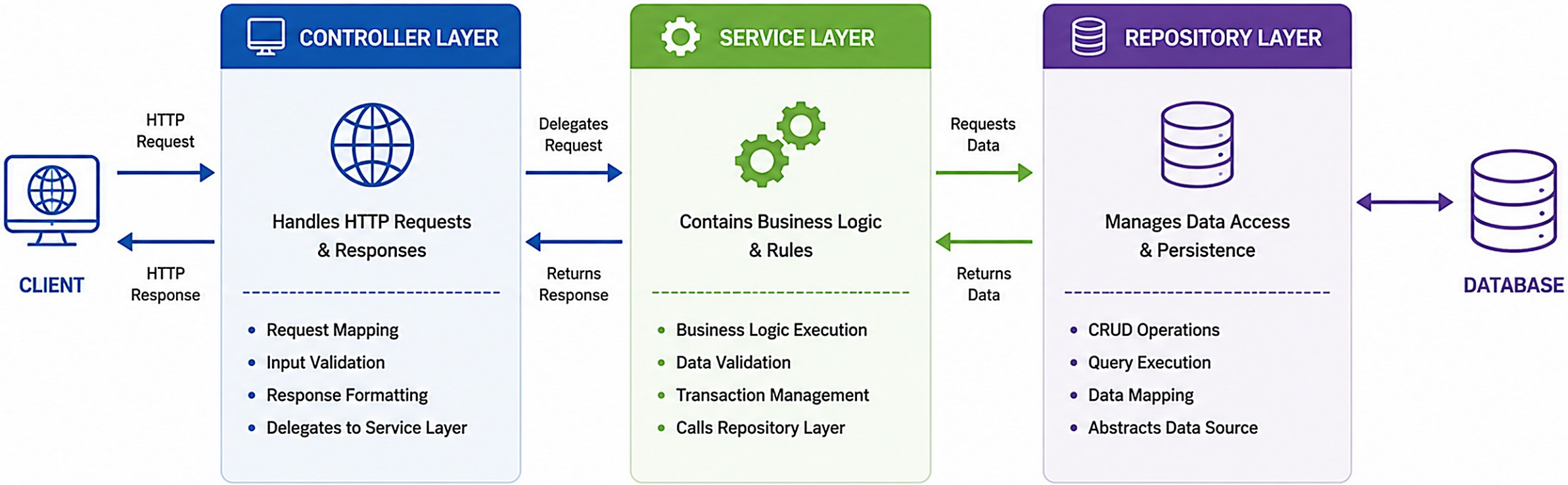
FLOW OF OPERATION

1. **Client** — Sends Request
2. **Controller** — Validates
3. **Service** — Processes
4. **Repository** — Interacts w/ DB
5. **Service** — Returns Result
6. **Controller** — Responds

KEY TAKEAWAY Each layer talks only to its neighbor -- Controller to Service, Service to Repository -- never skipping a layer.

Controller – Service – Repository (CSR) Pattern

A proven architectural pattern that separates concerns and promotes maintainability.



KEY BENEFITS

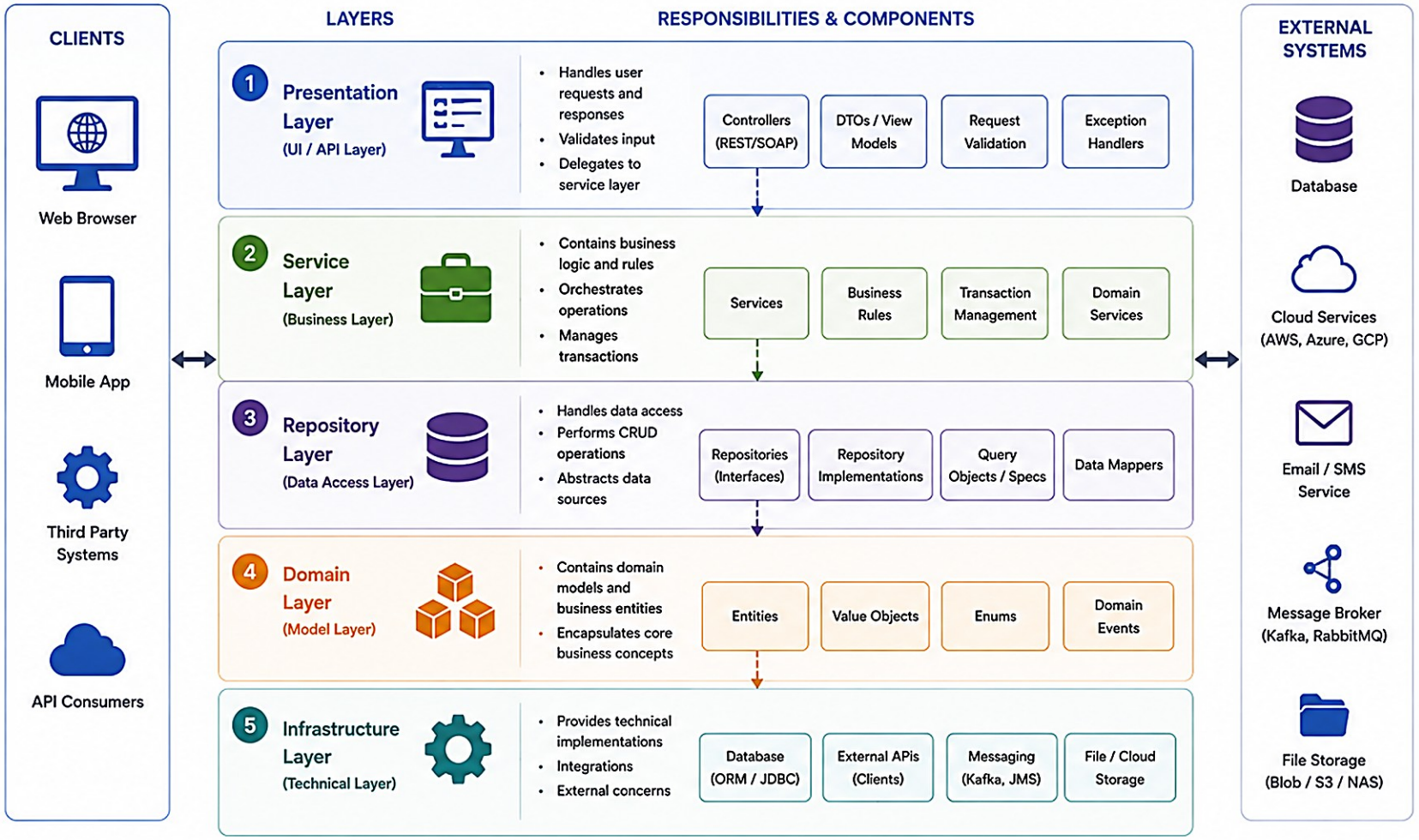
SEPARATION OF CONCERNS
Each layer has a distinct responsibility.

MAINTAINABILITY
Changes in one layer have minimal impact on others.

TESTABILITY
Layers can be unit tested independently.

Layered Application Architecture

A separation of concerns design that promotes maintainability, testability and scalability.



REQUEST FLOW

1. Client sends request to Controller
2. Controller calls Service
3. Service uses Repository
4. Repository accesses Domain & Data
5. Response flows back to Client

KEY BENEFITS

- **Separation of Concerns** — Each layer has a specific responsibility
- **Maintainability** — Changes in one layer have minimal impact on others
- **Testability** — Layers can be tested in isolation
- **Scalability** — Easy to extend and integrate
- **Reusability** — Business logic and components can be reused

KEY PRINCIPLES

- Loose Coupling
- High Cohesion
- Dependency Inversion
- Single Responsibility

CROSS-CUTTING CONCERNS (APPLICABLE TO ALL LAYERS)

Security (Authentication, Authorization) | Logging & Monitoring | Exception Handling | Performance Management | Configuration Management | Auditing & Tracing

CONTROLLER-SERVICE-REPOSITORY PATTERN (2 OF 2)

Layer responsibilities in detail, and why enterprise teams standardize on this pattern.

CONTROLLER

- Expose REST endpoints.
- Validate request data.
- Handle authentication/authorization.
- Return appropriate responses.

SERVICE

- Implement business logic.
- Apply business rules.
- Manage transactions (if needed).
- Call repository for data operations.

REPOSITORY

- Perform CRUD operations.
- Use ORM / SQL / query APIs.
- Hide data access complexity.
- Return domain data to Service.

WHY THIS PATTERN?

Separation of Concerns

Each layer changes for one reason only.

Easier Maintenance

Refactoring one layer rarely breaks another.

Improved Testability

Each layer can be tested in isolation.

Scalability & Flexibility

Layers scale and evolve independently.

Better Collaboration

Teams work across layers with clear contracts.

KEY TAKEAWAY A well-structured application today leads to an enterprise-ready system tomorrow.

SEPARATION OF CONCERNS

Separation of Concerns (SoC) is a design principle that divides an application into layers, where each layer has a specific responsibility.

"Each layer should have one reason to change." -- Software Engineering Best Practice

Layer	Focuses On	Does NOT Do
Controller	Handling web requests and responses.	Does not contain business logic or data access code.
Service	Business logic and orchestration.	Does not handle HTTP or direct database access.
Repository	Data access and persistence.	Does not contain business rules or request handling.

BENEFITS OF SEPARATION OF CONCERNS

Maintainability

Changes in one layer have minimal impact on others.

Testability

Each layer can be tested independently in isolation.

Scalability

Layers can be scaled independently based on demand.

Flexibility

Easier to adapt to new requirements and technologies.

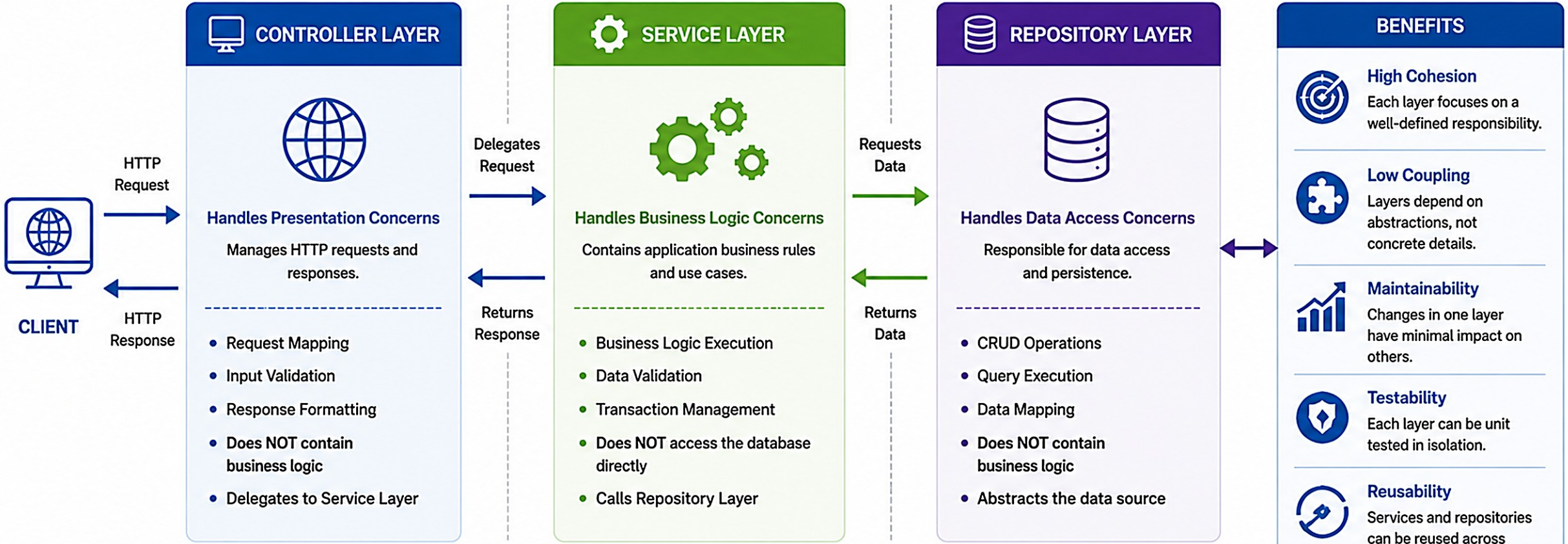
Team Productivity

Teams work on different layers in parallel.

KEY TAKEAWAY Separation of Concerns leads to clean code, lower complexity, and enterprise-ready applications.

Separation of Concerns

Each layer has a single responsibility and focuses only on its concern.



KEY TAKEAWAY | By separating concerns, we build systems that are easier to understand, test, extend, and maintain.

TRY IT YOURSELF — EXERCISE 1: LAYER BOUNDARY QUIZ

Reinforces: the Controller / Service / Repository boundaries from the Separation of Concerns table you just saw.

STEP	WHAT TO DO
Goal	Classify five CRM tasks -- HTTP mapping, uniqueness check, in-memory save, status transition, JSON serialization -- into Controller, Service, or Repository.
File	notes/layers.md (in examples/module-25-exercises/)
Classify	Record your five classifications, then check them against the reference mapping below.
Import Rule	Write, in your own words: controllers must not import repository types.
Fixtures	Seed plan: CUS-1001 = ACTIVE, CUS-1002 = PROSPECT -- the same fixtures used across this module.
Deliverable	Five correct classifications, the no-repository-import rule, and both fixtures named.

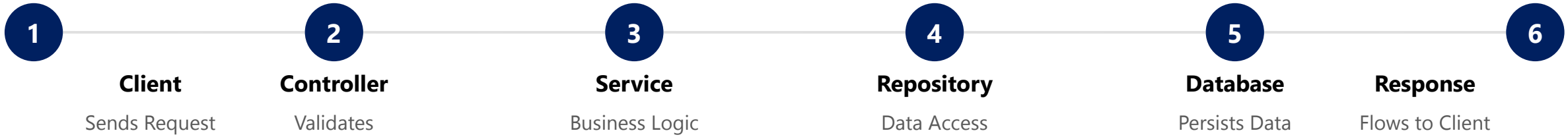
Reference -- Task -> Layer

```
Parse JSON / return ResponseEntity          -> Controller
PROSPECT -> ACTIVE status rule               -> Service
Map/store lookup by id                       -> Repository
Duplicate id rejection                        -> Service
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 1 (exercises/exercise-01-layer-boundaries.md): classify all five tasks and write the no-repository-import rule before continuing.

REQUEST PROCESSING FLOW

A well-defined flow ensures every request moves through the application layers in a controlled and predictable manner.



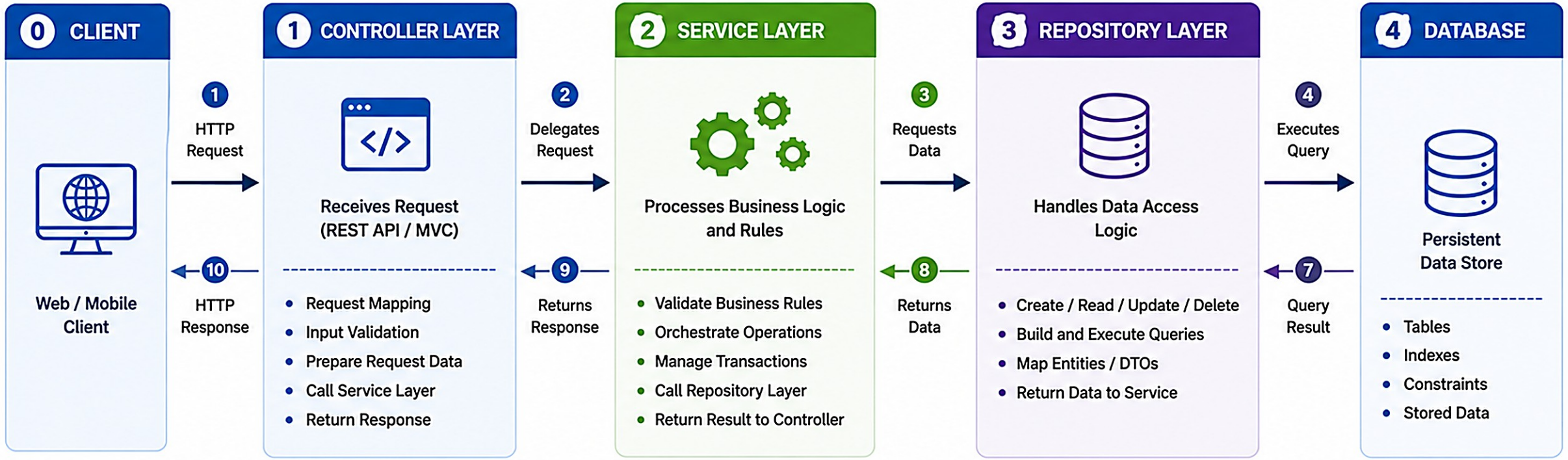
PER-LAYER RESPONSIBILITIES

Layer	What Happens Here
Controller	Receives HTTP request, validates input, authenticates/authorizes, maps to service call, returns response.
Service	Executes business rules, performs validations, orchestrates operations, manages transactions, calls Repository.
Repository	Abstracts data source complexity, executes CRUD operations, returns domain data to Service.
Database	Stores and manages application data.

KEY TAKEAWAY From client request to final response, each layer plays a specific role in delivering reliable business outcomes.

Request Processing Flow

A request flows through the application layers and returns a response to the client.



END-TO-END FLOW SUMMARY

1. Client sends HTTP request
2. Controller validates and delegates to Service
3. Service processes business logic and requests data
4. Repository executes query on Database
5. Database returns query result
6. Repository returns data to Service
7. Service returns result to Controller
8. Controller returns HTTP response to Client

KEY BENEFIT | A clear and organized flow improves maintainability, testability, and scalability of the application.

TRY IT YOURSELF — EXERCISE 2: PACKAGE SKETCH

Reinforces: turning the Controller -> Service -> Repository pattern into real Java packages before writing any code.

STEP	WHAT TO DO
Goal	Sketch the com.northstar.crm package tree: controller, service, repository, model (and optional dto).
File	notes/package-tree.md (in examples/module-25-exercises/)
Place Types	Assign CustomerController, CustomerService, CustomerRepository, InMemoryCustomerRepository, and Customer to the right package.
SOAP Note	If a SOAP endpoint exists from Lab 24, it stays a thin adapter and still calls the same CustomerService -- never a second copy of the business logic.
JPA Readiness	Write one sentence: a later JPA-backed CustomerRepository implementation must keep the same service method signatures.
Deliverable	Four packages present, all five types correctly placed, and the JPA-readiness sentence.

com.northstar.crm -- package tree

```
com.northstar.crm/
  controller/    // CustomerController
  service/       // CustomerService
  repository/    // CustomerRepository, InMemoryCustomerRepository
  model/         // Customer
  dto/           // optional -- request/response DTOs
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 2 (exercises/exercise-02-package-sketch.md): draw the package tree and place all five types before moving on.

WHAT IS A SERVICE LAYER? (1 OF 2)

The Service Layer is the heart of the application -- it contains the business logic and orchestrates operations.

It acts as an intermediary between the Controller and the Repository layer, ensuring that business requirements are met before any data is accessed or modified.

KEY CHARACTERISTICS

Business Logic — encapsulates core business rules and computations.

Validation — validates inputs and enforces business constraints.

Orchestration — coordinates multiple operations and components.

Reusability — provides reusable operations across multiple controllers.

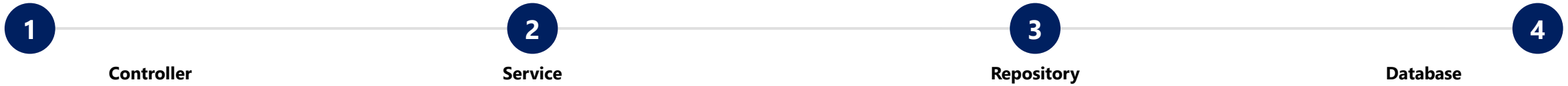
Transaction Management — manages transactions to ensure data consistency.

Abstraction — hides data access details from upper layers.

KEY TAKEAWAY The Service Layer holds the business rules and orchestrates work. It is independent of how data is stored and focuses only on what the business requires.

WHAT IS A SERVICE LAYER? (2 OF 2)

Where the Service Layer fits in the request flow, and a worked example: placing an order.



SCENARIO: PLACE ORDER

- Validate products and stock.
- Calculate order total and discounts.
- Apply business rules (e.g. credit limit).
- Create order and update inventory.
- Call Repository Layer to save data.
- Return result to Controller.

OrderService.java

```java
@Service
public class OrderService {
    public OrderResponse placeOrder(OrderRequest request) {
        // Business logic
        // Validation
        // Orchestration
        // Call repository
        return response;
    }
}
```

KEY TAKEAWAY A well-designed service layer makes your application clean, maintainable, and scalable.

BUSINESS LOGIC MANAGEMENT (1 OF 2)

The Service Layer is responsible for managing the business logic of the application.

It ensures that all business rules are applied consistently, transactions are coordinated, and data is used correctly to produce reliable results.

WHAT BELONGS IN BUSINESS LOGIC?

Category	Description
Business Rules	Implement domain rules (e.g. credit limit check).
Validations	Validate input and business constraints.
Calculations	Perform pricing, tax, discount, interest, and other calculations.
Workflow Orchestration	Coordinate multiple operations and services.
Transaction Management	Ensure atomicity and data consistency.

KEY TAKEAWAY Good business logic management leads to correct, consistent, and maintainable enterprise applications.

BUSINESS LOGIC MANAGEMENT (2 OF 2)

Worked example: a fund transfer service, and the benefits of managing business logic well.

EXAMPLE: FUND TRANSFER SERVICE

1. **Receive Request**
   Transfer A to B
2. **Apply Business Rules**
   Balance, limits, fraud
3. **Perform Calculations**
   Amount, fees, balance
4. **Orchestrate Ops**
   Debit A, credit B
5. **Return Result**
   Success or failure

BENEFITS OF EFFECTIVE BUSINESS LOGIC MANAGEMENT

Consistent Enforcement
Rules applied uniformly across the application.

Data Integrity
Reduces errors and ensures correct outcomes.

Better Testability
Business logic can be tested independently.

Reusability
Logic can be reused by multiple controllers/services.

Scalability
Supports future growth and complex business needs.

KEY TAKEAWAY Keep business logic in the Service Layer -- not in Controllers or Repositories -- for correct, consistent, and maintainable enterprise applications.

SERVICE DESIGN BEST PRACTICES (1 OF 2)

Well-designed services are maintainable, testable, scalable, and aligned with business needs.

#	Practice	What It Means
1	Single Responsibility	Each service should have one well-defined business responsibility.
2	Business Focused	Design services around business capabilities, not technical operations.
3	Loosely Coupled	Minimize dependencies between services; use interfaces.
4	High Cohesion	Keep related operations together in the same service.
5	Transaction Management	Use transactions appropriately; keep them short and within service boundaries.
6	Validate at the Service Layer	Enforce business rules and validate inputs before accessing data.
7	Exception Handling	Handle exceptions gracefully and throw meaningful business exceptions.
8	Stateless & Reusable	Design stateless services whenever possible to improve scalability.

KEY TAKEAWAY A good service is cohesive, focused on business value, independent of how data is stored, and easy to evolve.

SERVICE DESIGN BEST PRACTICES (2 OF 2)

A design checklist, plus a good and a bad service example, side by side.

SERVICE DESIGN CHECKLIST

- Does the service have a single, clear business purpose?
- Are service methods named based on business actions?
- Are input parameters validated?
- Does the service enforce business rules?
- Are repository dependencies injected (not created inside the service)?
- Are transactions defined at the right boundaries?
- Are exceptions logged and rethrown meaningfully?
- Is the service easy to unit test (mockable dependencies)?

Good: OrderService

```
placeOrder(request)
validateStock(request)
calculateTotal(request)
applyDiscount(request)
saveOrder(order)
```

Anti-Pattern: BadOrderService (avoid)

```
placeOrder()
sendEmail()
generateReport()
callPaymentGateway()
logToDatabaseDirectly()
```

KEY TAKEAWAY Follow these best practices to build clean, maintainable, scalable, and business-aligned services.

SERVICE DEPENDENCIES (1 OF 2)

A Service Layer does not work in isolation -- it depends on other components to perform its responsibilities.

Managing these dependencies properly leads to loosely coupled, flexible, and testable services.

WHERE SERVICE LAYER DEPENDENCIES COME FROM

Source	Examples
Repositories	Data access operations, CRUD and queries, persistence concerns.
Other Services	Reuse business capabilities, service-to-service calls, orchestration.
Configuration Properties	Application settings, feature flags, environment values.
Security Context	Authentication details, authorization checks, user/role information.
External Systems & Integrations	Third-party APIs, payment gateways, messaging systems.
Utility / Helper Components	Mappers/converters, validators, common utilities.

KEY TAKEAWAY Depend on abstractions, not on concrete implementations. Inject dependencies. Keep services focused and decoupled.

Service Dependencies

Services depend on other services through well-defined interfaces to promote reusability, testability and maintainability.

KEY PRINCIPLES

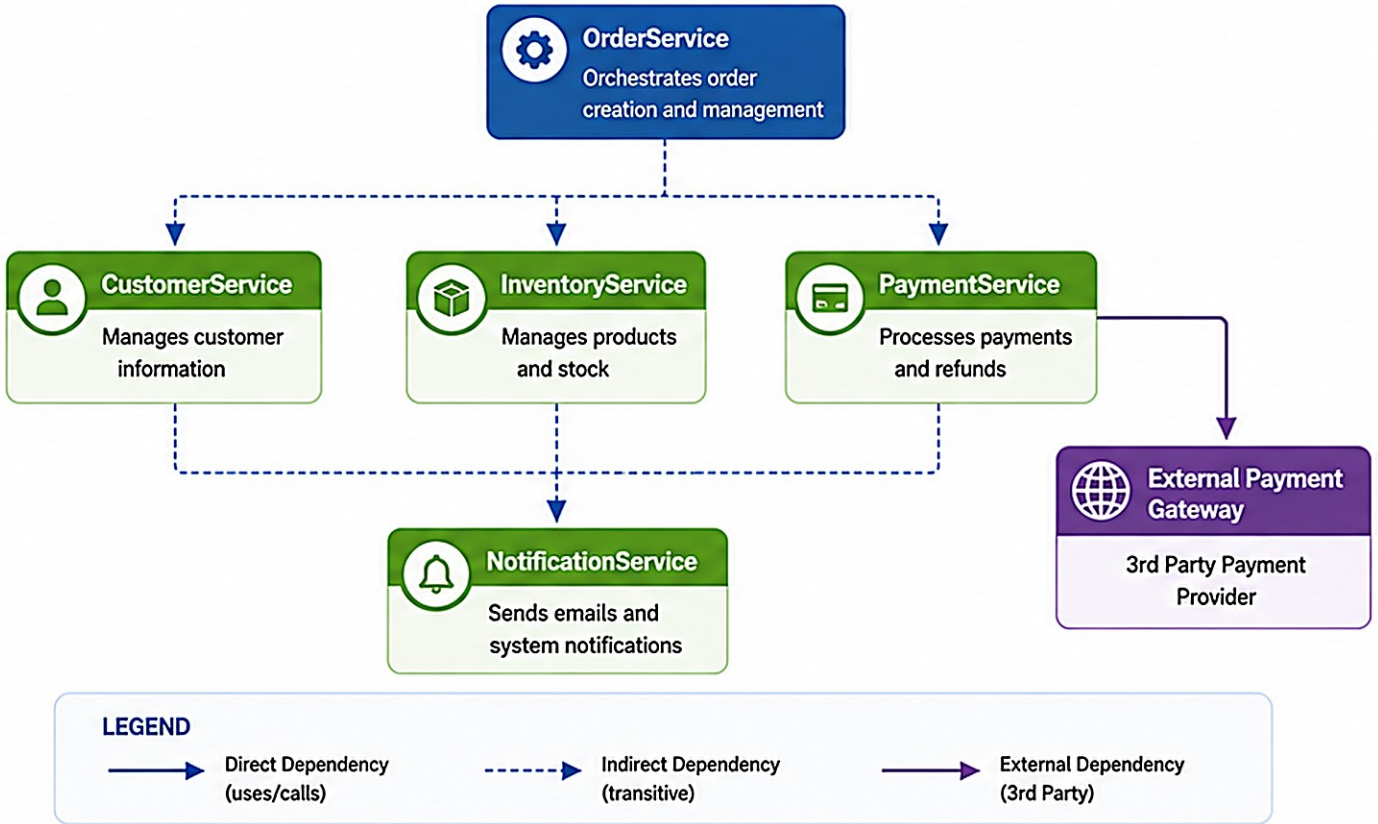
Abstraction
Depend on interfaces, not implementations.

Loose Coupling
Minimize dependencies between services.

High Cohesion
Each service focuses on a single responsibility.

Testability
Dependencies are easy to mock and test.

EXAMPLE: ORDER MANAGEMENT SYSTEM



BEST PRACTICES

Depend on Interfaces
Inject and use interfaces instead of concrete classes.

Use Constructor Injection
Prefer constructor injection for required dependencies.

Avoid Circular Dependencies
Keep the dependency graph acyclic.

Single Responsibility
Design services with a clear and focused responsibility.

Document Dependencies
Maintain clear documentation of service relationships.

KEY TAKEAWAY | Well-managed service dependencies lead to a flexible, scalable and maintainable architecture.

SERVICE DEPENDENCIES (2 OF 2)

Types of dependencies, best practices for managing them, and the impact of doing it well.

TYPES OF DEPENDENCIES

Mandatory — required for the service to function (e.g. Repositories).

Optional — used only in certain scenarios (e.g. NotificationService).

Runtime — resolved and used during execution (e.g. external APIs).

Interface-Based — the service depends on interfaces, allowing easy replacement and testing.

External — dependencies outside the application boundary (e.g. third-party APIs).

DEPENDENCY BEST PRACTICES

Depend on Abstractions — use interfaces for repositories, external clients, and other services.

Use Dependency Injection — inject dependencies via constructor (preferred).

Keep Loose Coupling — avoid tight coupling to concrete classes or static utilities.

Minimize Dependencies — include only what the service needs to fulfill its responsibility.

Design for Testability — easily mock dependencies in unit and integration tests.

Easier to Test | **Easier to Change** | **Improved Maintainability** | **Better Scalability** | **Higher Code Quality**

KEY TAKEAWAY Well-managed dependencies create clean, maintainable, and enterprise-ready service layers.

WHAT IS A REPOSITORY?

A Repository is a component in the Data Access Layer that abstracts and encapsulates data access logic.

It provides a clean, consistent API for performing CRUD operations and queries on data sources without exposing database details to the Service Layer.

WHAT A REPOSITORY DOES

Abstracts the Data Source — hides the database technology and query implementation.

Provides Data Operations — offers methods to create, read, update, delete, and query data.

Promotes Reusability — centralizes data access logic to avoid duplication.

Improves Maintainability — changes in data access logic are isolated within repositories.

Enhances Testability — repositories can be easily mocked for unit testing.

Analogy: Service <-> Repository <-> DB

```
// Service asks (like a customer at a store):
customerRepository.findByCustomerId("CUS-1001");

// Repository fetches it from storage
// (like store staff getting it from the back)
// -- the Service never needs to know how
//    or where the data is actually kept.
```

KEY TAKEAWAY A Repository provides a collection-like interface for data access, keeping business logic clean and persistence details hidden.

DATA ACCESS LAYER DESIGN (1 OF 2)

The Data Access Layer (Repository Layer) provides a clean abstraction between the application and the database technology.

DATA ACCESS LAYER ARCHITECTURE

1. **Service Layer**
   Calls repo interfaces
2. **Repository Interface**
   Defines the contract
3. **Repo Implementation**
   JPA, JDBC, etc.
4. **Data Source**
   Database, NoSQL, etc.

DESIGN PRINCIPLES

Principle	Description
Abstraction	Expose interfaces, not implementations. Shield the rest of the app from persistence details.
Single Responsibility	Handle only data access concerns, not business logic.
Loose Coupling	The Service Layer depends on repository interfaces, not concrete classes.
Technology Independence	Design repositories so switching databases is easier.
Consistency & Transactions	Ensure data integrity using proper transaction boundaries.

KEY TAKEAWAY Design the data access layer to isolate persistence details, promote reuse, and enable easy maintenance.

Data Access Layer Design

Designing a robust, maintainable and secure data access layer is key to a scalable enterprise application.

DESIGN PRINCIPLES

Abstraction
Hide data access implementation details from upper layers.

Single Responsibility
Handle only data access concerns.

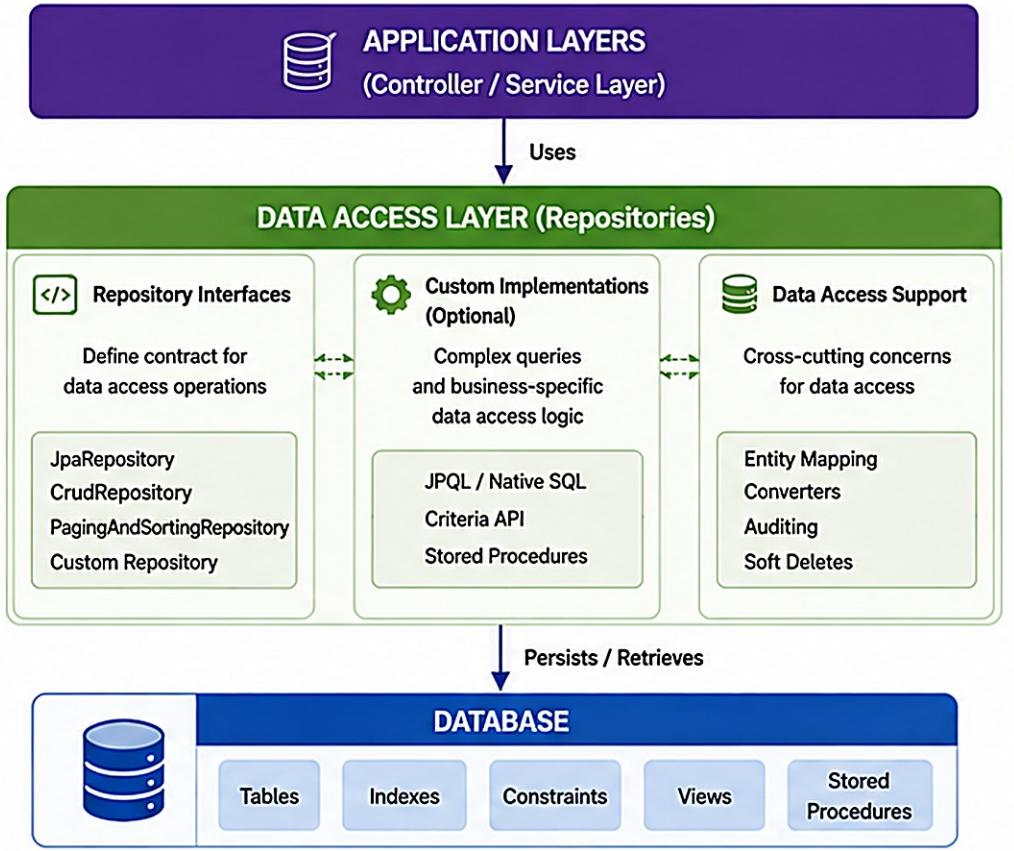
Consistency
Follow uniform patterns and naming conventions.

Reusability
Encapsulate common queries and logic.

Testability
Use interfaces and mocking for unit tests.

Security
Prevent SQL injection and expose only required data.

DATA ACCESS LAYER ARCHITECTURE



KEY RESPONSIBILITIES

CRUD Operations
Create, Read, Update, Delete data.

Query Execution
Execute optimized queries using JPQL / SQL.

Data Mapping
Map database rows to Entity / DTO objects.

Pagination & Sorting
Support efficient paging and sorting of data.

Transaction Support
Participate in transactions managed by the service layer.

Exception Translation
Convert database exceptions to meaningful application exceptions.

BEST PRACTICES

- Program to Interfaces, not Implementations.
- Use Spring Data JPA conventions.
- Keep queries in repositories, not in services or controllers.
- Use DTO projections to fetch only required data.
- Avoid N+1 query problem.
- Log slow queries and analyze performance.

KEY TAKEAWAY | A well-designed data access layer ensures performance, maintainability, security and flexibility.

DATA ACCESS LAYER DESIGN (2 OF 2)

Data Access Layer components, best practices, and a Spring Data JPA repository example.

DATA ACCESS LAYER COMPONENTS

Component	Description
Repository Interfaces	Define CRUD and query operations.
Queries	Derived queries, custom queries, or query builders.
Implementations	Concrete data access logic (JPA, JDBC, etc.).
Configuration	Data source, ORM settings, connection pooling.
Entity/Mapping	Map domain objects to database tables.
Exceptions & Translation	Translate persistence exceptions to application exceptions.

AccountRepository.java (Spring Data JPA)

```
public interface AccountRepository
    extends JpaRepository<Account, Long> {

  // Derived query method
  List<Account> findByCustomerId(Long id);

  @Query("SELECT a FROM Account a WHERE a.balance > :amount")
  List<Account> findAccountsWithBalanceGreaterThan(
      @Param("amount") BigDecimal amount);
}
```

BEST PRACTICES

- Use repository interfaces; program to interfaces for flexibility and testability.
- Keep repositories small and focused -- one repository per aggregate/root entity.
- Use meaningful method names; handle transactions at the service layer; avoid returning entities unnecessarily; log and handle data access errors.

KEY TAKEAWAY The service layer uses AccountRepository without knowing how data is accessed or stored -- a well-designed Data Access Layer is clean, maintainable, and easy to evolve.

REPOSITORY RESPONSIBILITIES

A Repository is the bridge between the Service Layer and the data source -- providing a clean, consistent, abstracted way to access data.

WHAT A REPOSITORY IS RESPONSIBLE FOR

Data Access Operations — provide methods to create, read, update, delete, and query data.

Query Abstraction — encapsulate query logic and hide the complexity of the underlying data source.

Data Mapping — map database records to domain entities (and vice versa).

Consistency & Integrity — ensure data integrity using transactions, constraints, and concurrency control.

Efficiency — optimize queries and use pagination, caching, and indexing when needed.

Exception Translation — convert low-level data exceptions into meaningful application exceptions.

WHAT A REPOSITORY SHOULD NOT DO

- Should not contain business logic or validations.
- Should not manage transactions (that's the Service Layer's job).
- Should not know about HTTP requests or responses.
- Should not log business events (use Service/Controller).
- Should not depend on specific database implementation details in the Service Layer.

REMEMBER

Repositories deal with how data is accessed and stored, not what the data means or what business rules apply.

KEY TAKEAWAY A well-defined repository layer leads to clean architecture, higher maintainability, and easier testing.

REPOSITORY BEST PRACTICES (1 OF 2)

Well-designed repositories make your application easier to maintain, test, and evolve.

#	Practice	What It Means
1	Program to an Interface	Define repository interfaces and depend on abstractions, not concrete implementations.
2	Keep Repositories Focused	One repository per aggregate root or entity. Avoid mixing unrelated data concerns.
3	Use Meaningful Method Names	Use names that clearly express intent (e.g. findByEmail, not get).
4	Favor Query Abstraction	Expose queries through method signatures, not raw persistence details.
5	Handle Transactions in the Service Layer	Repositories should not manage transactions -- leave that to the Service.

KEY TAKEAWAY A good repository hides data access complexity, promotes reuse, and keeps your business logic in the Service Layer.

REPOSITORY BEST PRACTICES (2 OF 2)

Practices 6-10, a do's and don'ts table, and a good repository interface example.

PRACTICES 6-10

6. Do Not Leak Persistence Details — never expose EntityManager, Session, or database-specific APIs to upper layers.

7. Return Domain Objects or DTOs — return entities or DTOs, not raw database rows or internal structures.

8. Optimize for Performance — use pagination, indexing, fetch joins, and caching where appropriate.

9. Write Testable Repositories — keep repositories small and use dependency injection to enable easy mocking.

10. Log and Handle Data Access Errors — log meaningful errors and translate low-level exceptions into application exceptions.

DO	DON'T
Use interfaces and DI	Put business logic in repositories
Keep methods small/specific	Return raw SQL results
Use parameter objects for complex queries	Expose persistence framework objects
Use pagination for large result sets	Catch and swallow exceptions

UserRepository.java (Good Example)

```
public interface UserRepository
    extends JpaRepository<User, Long> {

  // Derived query
  Optional<User> findByEmail(String email);

  // Custom query
  @Query("SELECT u FROM User u WHERE u.status = :status")
  List<User> findByStatus(
      @Param("status") UserStatus status);

  // Pagination
  Page<User> findByCreatedDateBetween(
      LocalDateTime start, LocalDateTime end,
      Pageable pageable);
}
```

KEY TAKEAWAY Follow these best practices to build reliable, maintainable, and high-quality data access layers.

TRY IT YOURSELF — EXERCISE 3: SERVICE LAYER SKELETON (STARTER)

STARTER exercise -- fill in the TODOs in a tiny Service + Repository demo, applying everything covered on the Repository Layer so far.

STEP	WHAT TO DO
Goal	Complete a TODO/fill-in-the-blank mini demo that rejects a duplicate CUS-1001 create.
Files	CustomerRepository.java, InMemoryCustomerRepository.java, CustomerService.java, LayerDemo.java under mini-src/com/northstar/crm/
Compile & Run	javac -d mini-out mini-src/com/northstar/crm/*.java, then java -cp mini-out com.northstar.crm.LayerDemo
Expected Output	duplicate blocked
Reflect	HTTP/Controller is intentionally absent here -- that arrives as Lab 25 starter work.

CustomerService.java -- fill in the blanks (STARTER, not a complete solution)

```
InMemoryCustomerRepository() {
  // TODO: seed CUS-1001 -> Amina Khan
  store.put(_____, _____);
}

void create(String id, String name) {
  // TODO: if exists, throw IllegalStateException("duplicate")
  if (repo._____(id)) throw new IllegalStateException("duplicate");
  repo.save(id, name);
}
```

KEY TAKEAWAY TRY IT NOW -- this is a STARTER: complete Exercise 3 (exercises/exercise-03-service-todo-skeleton.md), fill every blank, and confirm the console prints duplicate blocked.

CONTROLLER TO SERVICE FLOW

How an incoming request is processed from the Controller through the Service Layer and on to the Repository, and back.

FLOW EXAMPLE (GET /api/customers/123)

- 1. Client sends GET request to /api/customers/123.
- 2. Controller: validates the request, calls CustomerService.getById(123).
- 3. Service: executes business logic (e.g. checks permissions, applies rules).
- 4. Repository: calls CustomerRepository.findById(123).
- 5. Database: returns customer data.
- 6. Service: maps data to a response model.
- 7. Controller: returns a JSON response to the client.

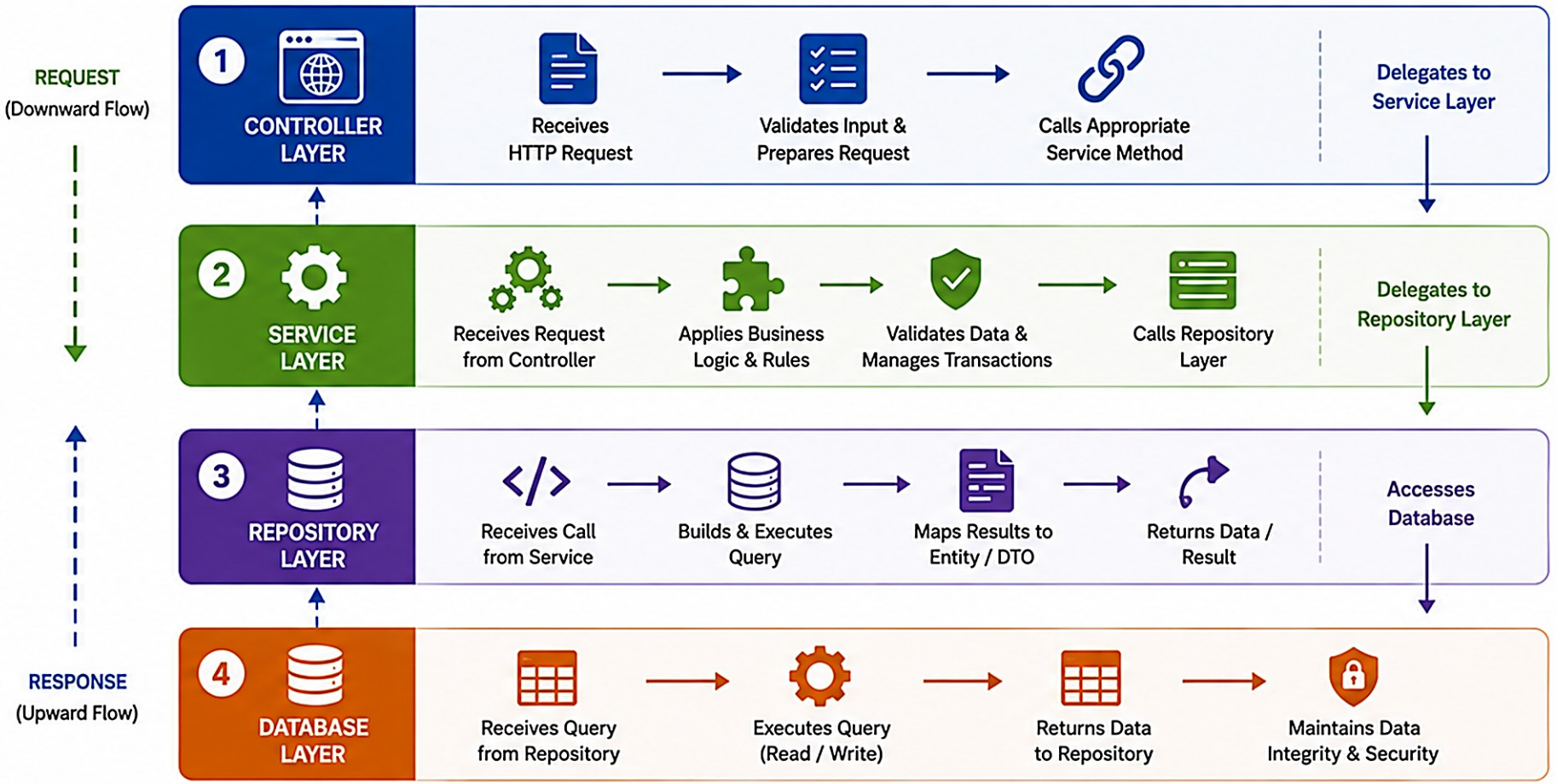
BEST PRACTICE REMINDERS

- Keep Controllers thin -- no business logic.
- Keep Services focused on business rules.
- Use Repositories only for data access.
- Validate input at the Controller level.
- Return consistent and meaningful responses.

KEY TAKEAWAY A clean flow from Controller to Service to Repository ensures scalable, maintainable, and testable applications.

Layer Communication Flows

Each layer has a clear responsibility and communicates only with the adjacent layer.



FLOW SUMMARY

1 Controller → Service
- Handles HTTP request
- Delegates to service
- Returns response to client

2 Service → Repository
- Contains business logic
- Manages transactions
- Requests data from repository

3 Repository → Database
- Executes queries
- Maps results
- Returns data to service

4 Database → Repository
- Stores and retrieves data
- Ensures data integrity
- Returns results

Request Flow (Downward)
Response Flow (Upward)

KEY TAKEAWAY | Layers communicate only with adjacent layers. This separation improves maintainability, testability, and scalability.

SERVICE TO REPOSITORY FLOW

The Service Layer uses the Repository Layer to perform data access operations, keeping business logic independent of how data is stored.

1. **Receives Request**
2. **Prepares Data**
3. **Calls Repository**
4. **Executes Access**
5. **Returns Result**

CustomerService.java (Spring Data JPA style)

```
@Service
public class CustomerService {
  private final CustomerRepository customerRepository;

  public CustomerService(CustomerRepository repo) {
    this.customerRepository = repo;
  }

  public CustomerDTO getCustomerDetails(Long id) {
    Customer c = customerRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Customer not found"));
    return CustomerDTO.fromEntity(c);
  }
}
```

BEST PRACTICES

- Depend on repository interfaces.
- Expose only necessary methods.
- Use meaningful method names.
- Keep repository calls in services, not controllers.
- Handle exceptions and map to business exceptions.
- Avoid returning entities when DTOs are sufficient.

KEY TAKEAWAY The Service Layer orchestrates business logic and relies on repositories for efficient, secure, and maintainable data access.

Service to Repository Flow

How the Service Layer interacts with the Repository Layer for data access

The Service Layer contains business logic and coordinates data access by calling Repository methods.

Purpose

Encapsulate business logic in Service Layer and data access in Repository Layer for clean separation of concerns.

1. SERVICE LAYER

CustomerService

1 Business Method Called
Controller calls a service method with input parameters.

2 Business Logic Execution
Service validates input and applies business rules.

3 Call Repository Method
Service calls the appropriate repository method to access data.

6 Process Result
Service processes the data returned from repository.

7 Return to Controller
Service returns result (DTO/Entity/List) to the controller.

Request
Method call with parameters

Response
Data returned from repository

2. REPOSITORY LAYER

CustomerRepository

4 Repository Method Invoked
Spring Data JPA repository method is invoked.

5 Database Operation
JPA/Hibernate generates SQL and executes the query.

6 Return Data
Query result is mapped to Entity/DTO and returned to the service.

EXAMPLE

```
// Service Layer
public List<Customer> getActiveCustomers() {
    return customerRepository.findByStatus("ACTIVE");
}

// Repository Layer (Spring Data JPA)
public interface CustomerRepository
      extends JpaRepository<Customer, Long> {
   List<Customer> findByStatus(String status);
}
```

KEY POINTS

- Service layer contains business logic.
- Repository layer handles data access.
- Service calls repository methods with parameters.
- Repository executes query and returns data.
- Service processes data and returns to controller.
- Promotes separation of concerns, testability and maintainability.

FLOW SUMMARY

1 Business Method Called → 2 Business Logic Execution → 3 Call Repository Method → 4 Repository Method Invoked → 5 Database Operation → 6 Return Data to Service → 7 Return Result to Controller

Best Practice

Keep business logic in Service Layer and data access in Repository Layer. Never access the database directly from the service or controller.

REPOSITORY TO DATABASE FLOW

The Repository Layer abstracts the database and handles the details of connecting, executing, and managing data operations.

1. **Repo Receives**
2. **Builds Query**
3. **Gets Connection**
4. **Executes Op**
5. **DB Returns**
6. **Maps to Service**

REPOSITORY RESPONSIBILITIES IN THIS FLOW

- Translate method calls into queries/commands, manage connections through the DataSource.
- Execute database operations and handle transactions (if configured at this level).
- Map results to domain objects or DTOs, and handle exceptions with meaningful translations.
- Return results to the Service Layer.

BEST PRACTICES

Connection Pooling | **Parameterized Queries** | **Fetch Only Required Data** | **Indexing & Pagination** | **Log Database Errors**

KEY TAKEAWAY The Repository Layer manages all database interactions, returning data to the Service Layer in a consistent and efficient way.

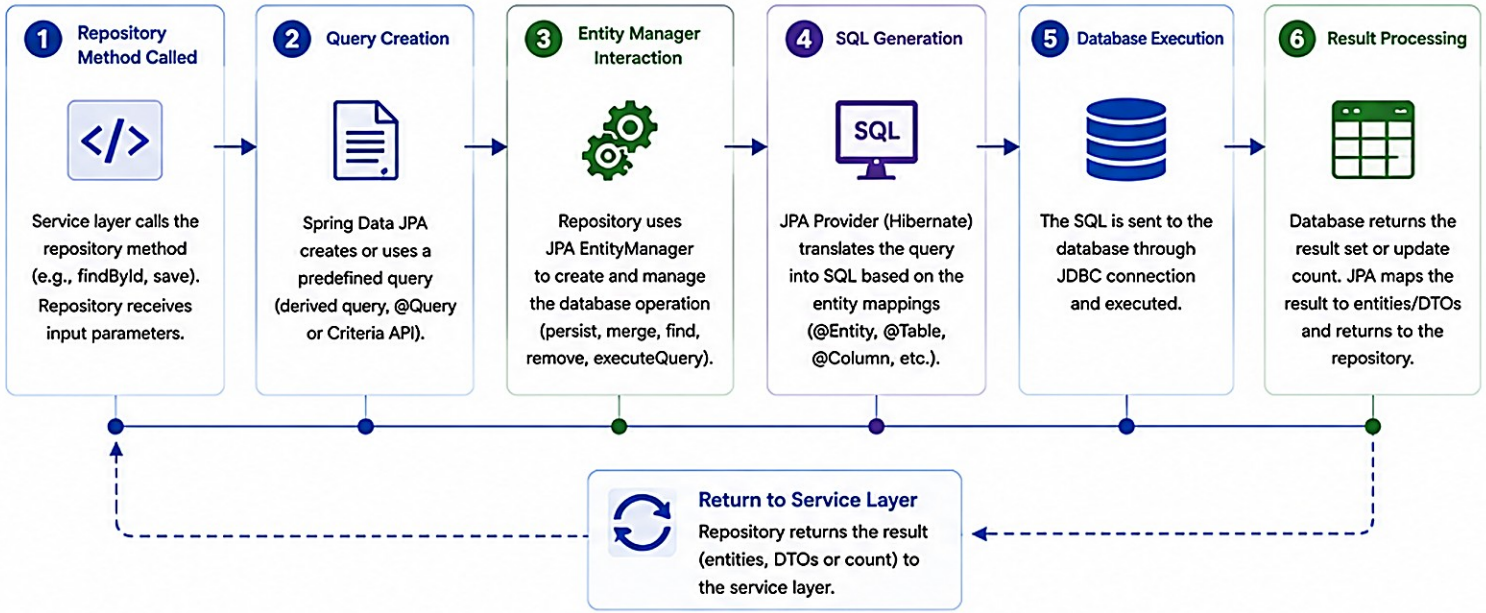
Repository to Database Flow

How the Repository Layer interacts with the Database for data persistence and retrieval

Purpose

Ensure efficient, secure, and reliable communication between the Repository Layer and the Database for data persistence and retrieval.

The **Repository Layer** abstracts database access and uses JPA/Hibernate to perform CRUD operations on the database.



EXAMPLE FLOW: findById(1L)

1. CustomerService calls customerRepository.findById(1L)
2. Spring Data JPA parses the method name and creates query: SELECT c FROM Customer c WHERE c.id = :id
3. EntityManager creates a query object and sets parameter (id = 1)
4. Hibernate generates SQL: SELECT * FROM customers WHERE id = ?
5. SQL is executed in the database and result is returned
6. Result set is mapped to Customer entity and returned to the repository

Repository returns Customer entity to the service layer

TECHNOLOGIES INVOLVED

- **Spring Data JPA** — Repository Abstraction
- **Spring Framework** — IoC & Transaction Management
- **HIBERNATE** — JPA Implementation (Provider)
- **Jakarta Persistence API** — Entity Management and Mapping
- **JDBC** — Database Connectivity and Communication
- **Database** — Data Storage and Retrieval

TYPES OF OPERATIONS

Create (save)	Read (find)	Update (update)	Delete (delete)
INSERT	SELECT	UPDATE	DELETE

KEY POINTS

- Repository layer hides database complexity from the service layer.
- JPA/Hibernate maps entities to tables and handles SQL generation.
- Use transactions (@Transactional) to ensure data consistency.
- Connection pooling and caching improve performance.

BEST PRACTICES

- Use repository interfaces for data access.
- Follow naming conventions for derived queries.
- Use pagination and projections for large datasets.
- Handle exceptions and optimize queries.
- Enable indexing and proper constraints in the database.

TRANSACTION BOUNDARIES (1 OF 2)

A transaction boundary defines the scope of work that must succeed or fail as a single unit.

Place the boundary in the Service Layer to ensure data consistency and maintain integrity.

Layer	Role in the Transaction	Notes
Controller	Handles HTTP requests only.	Never begins or manages a transaction.
Service (boundary)	Begins the transaction (@Transactional); coordinates repository calls.	Commits if all succeed, rolls back if any fail.
Repository	Executes data access; participates in the transaction.	Never annotated with @Transactional itself.
Database	Commits or rolls back at the end of the transaction.	Enforces durability and consistency.

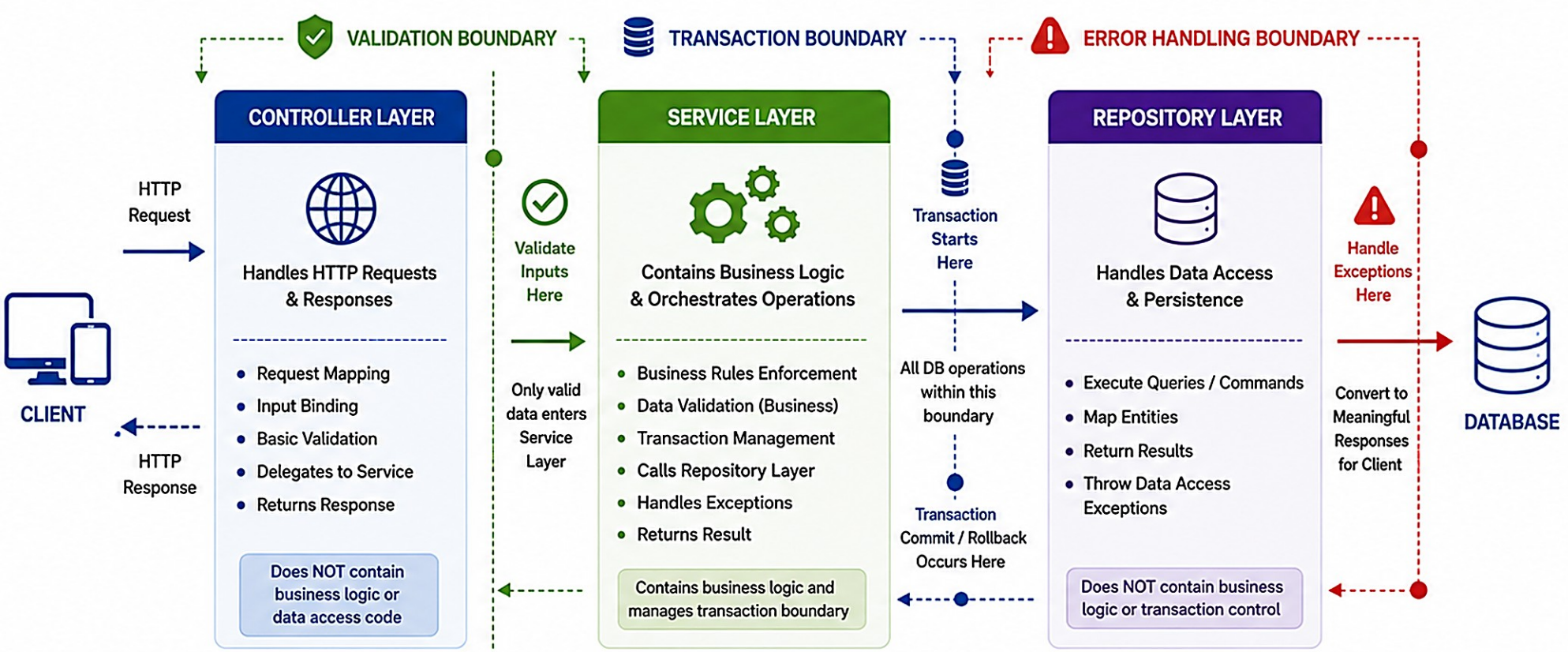
WHY THIS MATTERS

Data Consistency **Integrity Enforcement** **Simplified Error Handling** **Maintainability**

KEY TAKEAWAY The Service Layer owns the transaction boundary. Repositories participate in the transaction but never control it.

Transaction, Validation & Error Handling Boundaries

Clear boundaries ensure data integrity, correct validation and consistent error handling across layers.



LEGEND — Request Flow (Downward) · Response Flow (Upward) · Validation Boundary · Transaction Boundary · Error Handling Boundary

KEY GUIDELINES

Validation Boundary
Validate as early as possible (in Controller & Service).
Only valid data flows inward.

Transaction Boundary
Start transaction in Service layer, before first data access.
Commit only on success.
Rollback on any failure.

Error Handling Boundary
Catch exceptions in Service layer.
Convert to meaningful, consistent responses.
Never expose internal errors to client.

Outcome
Ensures data integrity, maintainability, and a consistent client experience.

KEY TAKEAWAY | Well-defined boundaries for validation, transactions and errors lead to secure, reliable and maintainable enterprise applications.

TRANSACTION BOUNDARIES (2 OF 2)

Do's and don'ts for placing transaction boundaries, and a working @Transactional example.

DO'S

- Define transaction boundaries in Service Layer methods.
- Use @Transactional at the Service layer.
- Keep transactions short and focused on a single business use case.
- Let repositories be simple data access components.
- Handle exceptions and let the transaction rollback automatically.

DON'TS

- Do not annotate repository methods with @Transactional (not required).
- Do not start a transaction in the Controller.
- Do not keep transactions open while calling external services.
- Do not perform long-running operations inside a transaction.
- Do not catch and swallow exceptions inside the transaction.

AccountService.java

```
@Service
public class AccountService {
  @Transactional
  public void transfer(Long fromId, Long toId, BigDecimal amount) {
    Account from = accountRepository.findById(fromId).orElseThrow(...);
    Account to = accountRepository.findById(toId).orElseThrow(...);
    from.debit(amount);   // may throw exception
    to.credit(amount);    // may throw exception
    accountRepository.save(from);
    accountRepository.save(to);
    // If any exception occurs, the transaction rolls back automatically
  }
}
```

KEY TAKEAWAY Define transaction boundaries in the Service Layer to ensure data consistency, integrity, and reliable business operations.

VALIDATION BOUNDARIES (1 OF 2)

Validation ensures data is correct, complete, and within allowed rules before it is processed or persisted.

Validation should occur at the appropriate layer to maintain separation of concerns and provide meaningful feedback.

Layer	What It Validates
Controller	Basic input sanitation (required fields, format). Rejects malformed requests early.
Service (primary boundary)	Business rules and use cases. Ensures data is valid for the domain.
Repository	Enforces data constraints at the persistence level. Ensures integrity and consistency.
Database	Enforces schema constraints and data integrity rules as a final safeguard.

WHY IT MATTERS

Improves Data Quality **Enforces Business Rules** **Reduces Errors** **Enhances Maintainability**

KEY TAKEAWAY Validate early at the Service Layer for business rules, and use Repository constraints for data integrity as a final safeguard.

Validation Boundaries

Define Where and How Validation Happens in a Layered Architecture

Purpose
Ensure data is valid at every boundary to maintain data integrity, security, and system reliability.

Validation should occur at multiple boundaries in an application to catch invalid data early, protect the system, and provide meaningful feedback.

1. PRESENTATION LAYER (UI / API)

Receives input from users or external systems.

Validation Performed
- Input format
- Required fields
- Data type
- Basic constraints
- Request size limits

Tools / Technologies
- HTML5 Validation
- JavaScript Validation
- Bean Validation (Controller)
- API Gateway / WAF

1 Input Validation Boundary

2. CONTROLLER LAYER

Handles requests and delegates to service layer.

Validation Performed
- @Valid / Bean Validation
- Path variables
- Request parameters
- Business rules (basic)

Tools / Technologies
- Jakarta Bean Validation
- Custom Validators
- @Validated
- Method Validation

2 Service Boundary

3. SERVICE LAYER

Contains business logic and orchestrates data operations.

Validation Performed
- Business rule validation
- Cross-field validation
- State validation
- Permission checks

Tools / Technologies
- Custom Validation
- Domain Validators
- Specifications
- Business Rules Engine

3 Repository Boundary

4. REPOSITORY LAYER

Interacts with database to persist or retrieve data.

Validation Performed
- Data integrity checks
- Existence checks
- Unique constraints
- Permission integrity

Tools / Technologies
- JPA / Hibernate
- DB Constraints
- Unique Indexes
- Triggers (if needed)

4 Database Boundary

5. DATABASE LAYER

Stores and manages application data.

Validation Performed
- Schema constraints
- NOT NULL / CHECK
- Foreign keys
- Unique constraints
- Data type constraints

Tools / Technologies
- RDBMS Constraints
- Stored Procedures
- Triggers
- Views

Cross-Cutting Concerns
Logging • Exception Handling • Security • Auditing

TYPES OF VALIDATION

Syntax Validation
Checks data format, type, length, pattern, etc.

Semantic Validation
Ensures data makes sense in the business context.

Business Rule Validation
Validates rules specific to business logic.

Data Integrity Validation
Ensures data consistency in the database.

BEST PRACTICES

- Validate as early as possible.
- Do not trust client-side validation alone.
- Use consistent error responses.
- Centralize validation logic where possible.
- Use validation groups for different scenarios.
- Log validation failures for monitoring.
- Keep validation messages user-friendly.

KEY POINTS

- Multiple validation boundaries ensure defense in depth.
- Validate at the right layer based on the type of validation.
- Service layer enforces business rules; repository ensures data integrity.
- Database constraints are the last line of defense for data correctness.
- Consistent validation leads to better user experience and maintainable code.

VALIDATION BOUNDARIES (2 OF 2)

Do's and don'ts for validation placement, and a service-layer validation code example.

DO'S

- Validate at the Service Layer for business rules.
- Use Repository/database constraints for data integrity.
- Return clear, actionable error messages.
- Keep validation logic close to where data is used.
- Use reusable validation rules; log validation failures for monitoring.

DON'TS

- Don't rely only on database constraints.
- Don't duplicate validation logic across many layers inconsistently.
- Don't ignore edge cases and null values.
- Don't expose internal validation details to clients.
- Don't allow invalid data to reach persistence.

CustomerService.java (Service-Layer Validation)

```
public class CustomerService {
  public CustomerDto createCustomer(CreateCustomerRequest request) {
    // Business validation
    if (isBlank(request.getEmail())) throw new ValidationException("Email is required.");
    if (request.getAge() < 18)
      throw new ValidationException("Customer must be at least 18 years old.");
    // ...other business rules
    return repository.save(customer);
  }
}
```

KEY TAKEAWAY Validate early, fail fast, and keep data clean -- ensuring reliability and trust across your application.

ERROR HANDLING BOUNDARIES (1 OF 2)

Error handling boundaries define where exceptions are caught, handled, logged, and translated into meaningful responses.

Place boundaries at the right layers to prevent leakage of internal details and to provide a consistent experience for API consumers.

Layer	Error Handling Role
Controller (API boundary)	Catches exceptions from lower layers; translates to API error responses; returns proper HTTP status codes.
Service (business boundary)	Handles business rule violations, adds context and meaningful messages, throws custom business exceptions.
Repository (data access boundary)	Handles data access errors; wraps low-level exceptions in data exceptions; does not return nulls.
Database (persistence boundary)	Enforces constraints; may raise database errors (e.g. unique key, foreign key violations).

WHY IT MATTERS

Protects Internal Details **Consistent Client Experience** **Easier Troubleshooting** **Better Maintainability**

KEY TAKEAWAY Catch errors where you can recover or add context. Translate them into clean, consistent error responses at the boundary that communicates with the client.

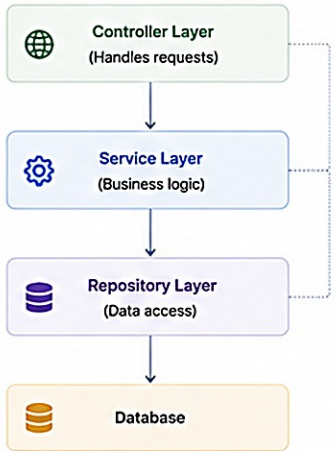
Copilot to scaffold service/repository structures

Use GitHub Copilot to quickly generate clean, consistent, and best-practice implementations for your Service and Repository layers.

Why use Copilot?

- Speeds up development by scaffolding boilerplate code
- Follows best practices and naming conventions
- Reduces repetitive coding and human errors
- Lets you focus on business logic and design

Typical Layered Architecture



Copilot in Action – Step by Step

1 Write a clear prompt (comment)

```
// Create a Spring Data JPA repository
// for entity Customer with basic CRUD
// and a custom method to find by email
public interface CustomerRepository
      extends JpaRepository<Customer, Long> {
   Optional<Customer> findByEmail(String email);
}
```

2 Copilot suggests the code AI Suggestion

```
public interface CustomerRepository
      extends JpaRepository<Customer, Long> {
   Optional<Customer> findByEmail(String email);
   List<Customer> findByActiveTrue();
```

3 Prompt for Service interface

```
// Create a service interface for Customer
// with methods for CRUD and business
// operations
public interface CustomerService {
   Customer create(CustomerDto dto);
   Customer getById(Long id);
   List<Customer> getAll();
   Customer update(Long id, CustomerDto dto);
   void delete(Long id);
   Optional<Customer> findByEmail(String email);
}
```

4 Copilot suggests the code AI Suggestion

```
public interface CustomerService {
   Customer create(CustomerDto dto);
   Customer getById(Long id);
   List<Customer> getAll();
   Customer update(Long id, CustomerDto dto);
   void delete(Long id);
   Optional<Customer> findByEmail(String email);
   boolean existsByEmail(String email);
}
```

5 Prompt for Service implementation

```
// Create a @Service class implementing CustomerService
// Use CustomerRepository inside. Add @Transactional
// and basic validations.
```

5 Copilot generates the implementation

Example: Generated Service Implementation

```
@Service
@RequiredArgsConstructor
@Transactional
public class CustomerServiceImpl implements CustomerService {
    private final CustomerRepository customerRepository;

    @Override
    public Customer create(CustomerDto dto) {
        if (customerRepository.existsByEmail(dto.email())) {
            throw new BusinessException("Email already exists");
        }
        Customer customer = Customer.builder()
                .name(dto.name())
                .email(dto.email())
                .active(true)
                .build();
        return customerRepository.save(customer);
    }

    @Override
    @Transactional(readOnly = true)
    public Customer getById(Long id) {
        return customerRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Customer not found"));

    // ... other methods generated by Copilot
}
```

Best Practices

- Write clear, specific prompts as comments.
- Review and refine generated code.
- Follow your project's naming and structure.
- Keep business logic in Service layer.
- Use Repository only for data access.
- Add validations and error handling.

Prompts Cheat Sheet

- "Create a Spring Data JPA repository for entity {Entity}"
- "Create a service interface for {Entity} with CRUD operations"
- "Create a @Service implementation for {Entity}Service"
- "Add validation and exception handling"
- "Add pagination and sorting support"

Pro Tip

Start with the interface, then generate the implementation.
Iterate with Copilot to add validations, transactions, logging, and custom business rules.

Goal: Use Copilot to accelerate scaffolding of clean, testable, and maintainable Service and Repository layers.

Error Handling Boundaries

Define Where Exceptions Are Handled and Translated Across Application Layers

Purpose
Ensure consistent error handling, prevent implementation leakage, and provide meaningful error responses to users while protecting system details.

Define clear boundaries for handling errors so that each layer is responsible for managing and translating exceptions appropriately.

1. PRESENTATION LAYER (UI / API)

Handles user interaction and formats error responses.

Responsibilities
- Handle and display errors to users
- Return proper HTTP status codes
- Log request and response errors
- Do not expose internal details

Error Types Handled
- Validation errors
- Authentication errors
- Authorization errors
- Business errors (translated)
- System unavailable errors

1 Presentation Boundary

2. CONTROLLER LAYER

Validates requests and delegates to service layer.

Responsibilities
- Validate request data
- Handle binding / format errors
- Convert exceptions to meaningful responses
- Delegate business logic exceptions

Error Types Handled
- Input/Validation errors
- Missing/Invalid parameters
- Method argument errors
- Resource not found (quick checks)

2 Controller Boundary

3. SERVICE LAYER

Contains business logic and orchestrates operations.

Responsibilities
- Handle business rule violations
- Manage transactions
- Translate repository exceptions to business exceptions
- Decide what to retry or compensate

Error Types Handled
- Business rule violations
- State transition errors
- Concurrency errors
- External service errors
- Transaction failures

3 Service Boundary

4. REPOSITORY LAYER

Performs data access and persistence operations.

Responsibilities
- Handle data access exceptions
- Translate to meaningful data layer errors
- Do not handle business logic errors
- Do not expose SQL/ORM details

Error Types Handled
- Data access errors
- Constraint violations
- Entity not found
- Optimistic lock failures
- Connection / Timeout errors

4 Repository Boundary

5. DATABASE LAYER

Stores and manages data.

Responsibilities
- Enforce constraints
- Ensure data integrity
- Return specific database error codes

Error Types Handled
- Constraint violations
- Deadlocks
- Timeout
- Connection failures
- Low-level DB errors

ERROR FLOW ACROSS BOUNDARIES

Error Occurs
Exception is thrown in a layer

Handled at Boundary
Catch and handle at the defined boundary

Translate Exception
Convert to appropriate exception for the next layer

Propagate Upward
Continue until it reaches a layer that can handle it

Respond to Client / User
Return meaningful response without leaking internals

BEST PRACTICES

- Define clear error handling responsibilities for each layer.
- Handle exceptions at the right boundary, not everywhere.
- Translate exceptions; do not leak internal implementation details.
- Use custom exception hierarchy for business and application errors.
- Provide consistent and user-friendly error responses.
- Log all errors with sufficient context and correlation IDs.
- Monitor error rates and set up alerts.

KEY TAKEAWAYS

- Handle errors at the correct boundary to maintain separation of concerns.
- Translate exceptions between layers to protect implementation details.
- Provide meaningful errors to users while ensuring system security.
- Let lower layers throw exceptions; higher layers decide how to handle them.
- Consistent error handling improves reliability, maintainability, and user experience.

Example
ConstriantViolationException (DB) → *DataIntegrityException* (Repository) → *DuplicateResourceException* (Service) → *409 Conflict* (Controller/API) → User-friendly error response (JSON).

ERROR HANDLING BOUNDARIES (2 OF 2)

Do's and don'ts for error handling placement, and a Spring global exception handler example.

DO'S

- Define and document an error handling strategy per layer.
- Catch specific exceptions and add context.
- Use custom exceptions for business errors.
- Log errors with correlation IDs.
- Return meaningful, consistent messages to clients; fail fast and avoid swallowing exceptions.

DON'TS

- Don't return stack traces to clients.
- Don't catch and ignore exceptions.
- Don't expose internal or database error codes.
- Don't use generic Exception catch blocks.
- Don't return nulls from repositories; don't mix error handling and business logic.

GlobalExceptionHandler.java (Spring)

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(InvalidAmountException.class)
  public ResponseEntity<ErrorResponse> handleInvalidAmount(InvalidAmountException ex) {
    return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR", ex.getMessage()));
  }
  // ...other @ExceptionHandler methods map each business exception to a status code
}
```

KEY TAKEAWAY Handle errors at the right boundaries, log with context, and return meaningful responses -- no surprises to your clients.

TRY IT YOURSELF — EXERCISE 4: AI REVIEW POLICY

Reinforces: the AI review discipline behind Lab 25's Security/AI Review grading category (10 of 100 marks).

STEP	WHAT TO DO
Goal	Document accept/reject criteria for AI (e.g. Copilot) drafts before Lab 25 begins.
File	notes/ai-review-policy.md -- header it with correlation id lab25-001.
Accept/Reject Rows	Copy the four reference rows below, then add one row of your own.
Manual Fallback	Note that if Copilot is unavailable, mark N/A and complete the layering manually -- AI use is optional, never required.
Boundary	This exercise writes policy only -- it does not generate production code via AI.
Deliverable	The lab25-001 header, at least four accept/reject rows, and the manual-fallback note.

Reference -- AI Suggestion -> Verdict

```
Service returns ResponseEntity              -> Reject
Controller calls Map store directly          -> Reject
Service uses repository interface            -> Accept after review
Hard-coded prod password                     -> Reject
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 4 (exercises/exercise-04-ai-review-policy.md): write your lab25-001 AI review policy before Lab 25 begins.

TRY IT YOURSELF — EXERCISE 5: SERVICE TEST PLAN

Reinforces: writing testable services (Repository Best Practice #9) ahead of Lab 25's graded CustomerServiceTest.

STEP	WHAT TO DO
Goal	List at least three service-level test cases Lab 25's CustomerServiceTest must prove.
File	notes/service-test-plan.md (in examples/module-25-exercises/)
Cases	Get seeded Amina Khan (CUS-1001); duplicate create fails; missing id fails; (optional) PROSPECT -> ACTIVE.
Fake vs In-Memory	State that a fake repository or the real InMemoryCustomerRepository is equally acceptable for unit tests.
Dual Green	Note 'mvn test green twice' as a lab habit to build now -- not pre-lab work to run yet.
Boundary	Do not write the full JUnit class here -- plan only.

service-test-plan.md -- case list (excerpt)

```
1. getSeededCustomer_returnsAminaKhan()
2. createDuplicateId_throwsIllegalStateException()
3. getMissingId_throwsNotFoundException()
4. (optional) transitionProspectToActive_updatesStatus()
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 5 (exercises/exercise-05-test-plan.md): list your test cases before writing any JUnit code in the lab.

TRY IT YOURSELF — EXERCISE 6: LAB 25 READINESS CHECKLIST

Capstone pre-lab check -- confirm your Boot CRM baseline and scope boundary before the timed Lab 25 begins.

STEP	WHAT TO DO
Goal	Confirm your Northstar CRM baseline is ready and the Lab 25 scope boundary is clear before the timed lab starts.
File	notes/lab25-readiness.md (in examples/module-25-exercises/)
Path	Write the project path: examples/lab25-crm.
Depends On	Confirm Labs 23-24's Boot CRM (+ optional SOAP) already compiles and runs.
Evidence Plan	Plan where lab screenshots will go: notes/screenshots/lab-25/.
Defer	Explicitly defer transactions (Lab 27), security/JWT (Lab 28), and Bean Validation polish (Lab 29).

Scope boundary -- do not build later technology yet

```
Do now                                  Do not add yet
Controller -> Service -> Repository seam  JPA/PostgreSQL swap (Lab 39)
In-memory CustomerRepository + fixtures   @Transactional transfers (Lab 27)
Lifecycle/uniqueness rules in the service JWT role matrix (Lab 28)
AI review notes (lab25-001)               Bean Validation @Valid polish (Lab 29)
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 6 (exercises/exercise-06-lab25-readiness.md), the capstone check, right before Lab 25 begins.

LAB OVERVIEW -- SERVICE AND REPOSITORY LAYERS

Lab 25: Service and Repository Layers with AI Assistance -- Northstar CRM Layering

BUSINESS SCENARIO

- Controllers talking directly to storage create tangled Boot apps that cannot grow transactions, security, or persistence swaps cleanly.
- Leadership freezes the seam: every Customer read/write path is Controller -> Service -> Repository; in-memory storage is fine for now.
- Controllers never import repositories. AI-assisted drafts (Copilot or similar) require a dated, honest review log.

LEARNING OBJECTIVES

- Separate Controller, Service, and Repository responsibilities for Customer CRUD and status updates.
- Define a Spring Data-style CustomerRepository and an in-memory implementation.
- Enforce lifecycle rules (e.g. PROSPECT -> ACTIVE) in the Service, never the Controller.
- Write focused service unit tests with a fake or in-memory repository.
- Explain what changes when the repository later becomes JPA against PostgreSQL.

WHAT YOU BUILD

- Copy prior Boot CRM to lab25-crm; define CustomerRepository and a seeded InMemoryCustomerRepository; put rules in CustomerService; keep CustomerController thin (zero repository imports); wire create/list/status paths; write CustomerServiceTest; log AI review lab25-001; verify dual green mvn test.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; CUS-9999 proves not-found handling.

LAB TASKS AND DELIVERABLES

Northstar CRM Layering -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch prior Boot CRM to lab25-crm; confirm Customer/CustomerStatus compile.
2	Define the Spring Data-style CustomerRepository interface (no JPA yet).
3	Implement seeded InMemoryCustomerRepository (CUS-1001, CUS-1002).
4	Implement CustomerService with duplicate/not-found/transition rules.
5	Build a thin CustomerController -- prove zero repository imports.
6	Wire create, list, and duplicate rejection through all three layers.
7	Unit-test CustomerService with a fake/in-memory repository (no MVC needed).
8	Log AI review lab25-001 + JPA readiness note; run failure experiments + evidence pack.

EXPECTED DELIVERABLES

- Layered Controller -> Service -> Repository source.
- Seeded in-memory repo with CUS-1001 / CUS-1002.
- HTTP evidence + duplicate/not-found failures.
- CustomerServiceTest output; AI review notes (or manual N/A).
- Layering/JPA readiness notes; dual green mvn test; no secrets or target/ committed.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/AI Review 10 · Docs 10

KEY TAKEAWAY Controllers importing repositories lose core marks; passing tests with only assertNotNull loses verification/AI marks.

MODULE 25 SUMMARY

In this module, we explored how the Service and Repository layers work together to build scalable, maintainable, and testable enterprise applications.

KEY CONCEPTS COVERED

Layered Architecture

Controller, Service, Repository, and Database each own one responsibility.

Service Layer

Business logic, validation, orchestration, and transaction management.

Repository Layer

Clean, abstracted data access that hides persistence details.

Layer Communication

Controller to Service to Repository to Database, and back.

Enterprise Boundaries

Transaction, validation, and error handling boundaries.

Best Practices

Dependency injection, testability, and clean separation of concerns.

MODULE FLOW RECAP

1. **Controller (thin)**
2. **Service (rules)**
3. **Repository (data)**
4. **Boundaries**
5. **Lab: Northstar CRM**

KEY TAKEAWAY By correctly designing and using the Service and Repository layers, we build applications that are robust, testable, and ready for enterprise scale.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of the Service Layer, transactions, and the Repository Layer's core purpose.

1. What is the primary responsibility of the Service layer?

A. Handling HTTP requests
B. Implementing business logic
C. Accessing the database tables
D. Rendering the user interface

2. Which layer should contain @Transactional in a typical Spring Boot application?

A. Controller layer
B. Service layer
C. Repository layer
D. Database layer

3. What is the main purpose of the Repository layer?

A. Validate input data
B. Manage business rules
C. Abstract data access and interact with the database
D. Handle exceptions globally

4. Which annotation is commonly used in Spring Data JPA repositories?

A. @Service
B. @Repository
C. @Controller
D. @Component

KEY TAKEAWAY The Service layer owns business logic and transactions; the Repository layer owns data access -- never the reverse.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on validation boundaries, layering violations, request flow, and error handling.

5. At which boundary should input validation ideally occur?

A. Controller layer

B. Service layer

C. Repository layer

D. Database layer

6. What happens if the Service layer directly uses JDBC or EntityManager instead of a Repository?

A. It improves performance

B. It violates separation of concerns

C. It reduces code readability

D. It is required by Spring Boot

7. Which flow correctly represents the processing path of a request?

A. Controller -> Repository -> Service -> Database

B. Controller -> Service -> Repository -> Database

C. Repository -> Controller -> Service -> Database

D. Service -> Controller -> Database -> Repository

8. Where should error handling and exception translation ideally be centralized?

A. In the Repository layer

B. In the Service layer

C. In the Controller layer

D. In the Database layer

KEY TAKEAWAY Validate business rules at the Service layer, never bypass the Repository abstraction, and centralize error translation at the Controller boundary.

KNOWLEDGE CHECK (3 OF 3)

Challenge Question: Why is it important to keep the Controller layer thin?

CHALLENGE QUESTION (FROM THE CURRICULUM)

"Why is it important to keep the Controller layer thin?" -- write 3-5 sentences in your own words before revealing the model answer below.

MODEL ANSWER

Separation of Concerns — the Controller's one job is HTTP in, HTTP out; mixing in business logic gives it a second reason to change.

Testability — business rules trapped in a Controller require a full web/MVC test setup instead of a fast, isolated unit test.

Reusability — a thin Controller lets the same business logic be reused by another Controller, a SOAP endpoint, or a batch job -- as Module 24's SOAP endpoint reuses this exact CustomerService.

Enforceability — a thin Controller with zero repository imports is exactly what Lab 25 requires and instructor-verifies by reading the import statements.

KEY TAKEAWAY Understanding the roles, flows, and boundaries of Service and Repository layers helps you build scalable, maintainable, enterprise-ready applications.

"Thin controllers, rule-owning services, and data-hiding repositories -- the backbone of every enterprise Spring app."

MODULE 25 COMPLETE

Service and Repository Layers

You can now design, build, and test layered Spring applications -- with business logic in the Service, data access in the Repository, and clean boundaries for transactions, validation, and errors.